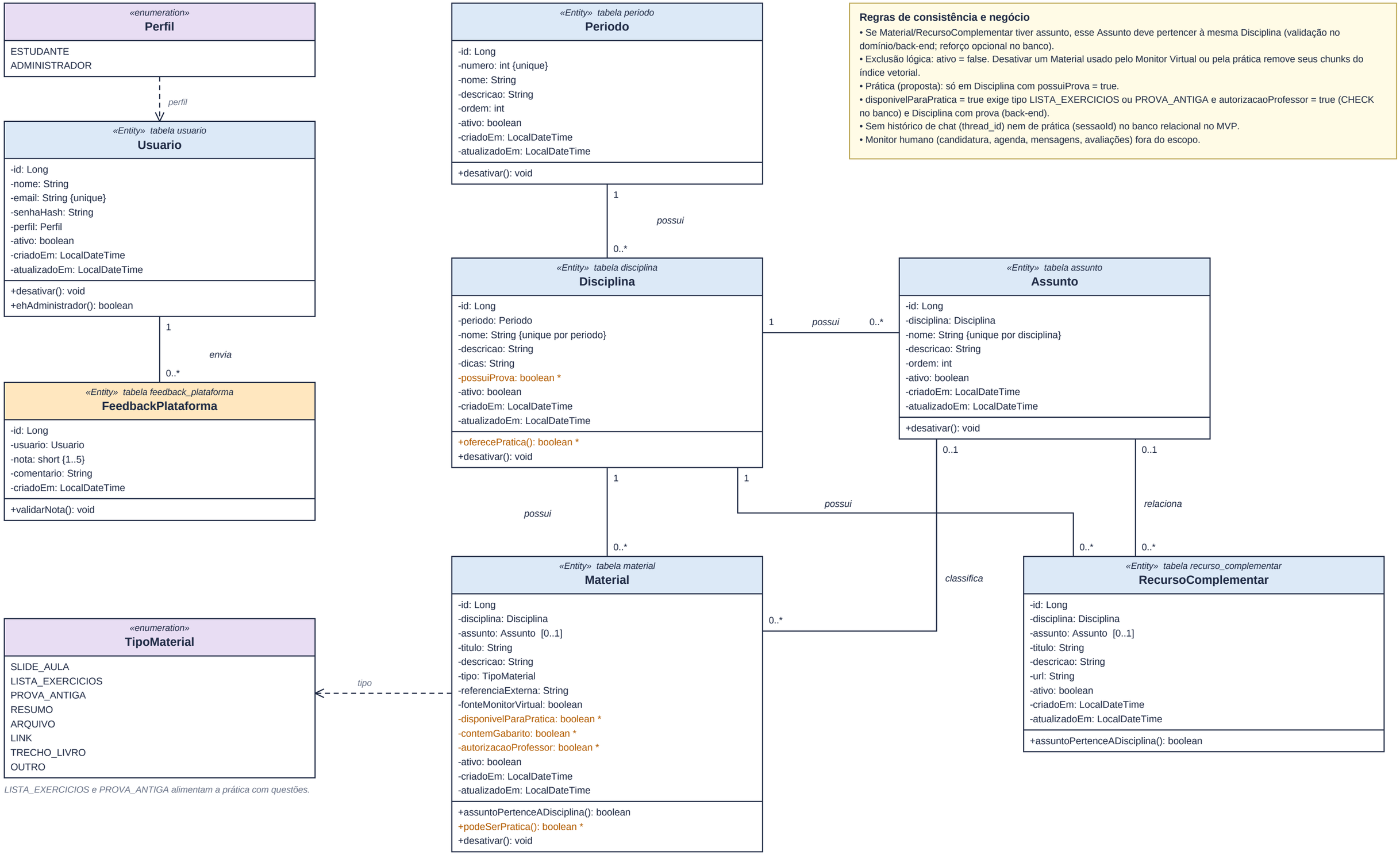
NEURON — Diagrama de Classes: Modelo de Domínio

Entidades JPA (Spring Boot) mapeadas para PostgreSQL. Associações com multiplicidades conforme a modelagem de dados.

classe nova (proposta) item laranja* = proposta a validar



LISTA_EXERCICIOS e PROVA_ANTIGA alimentam a prática com questões.

Regras de consistência e negócio

- Se Material/RecursoComplementar tiver assunto, esse Assunto deve pertencer à mesma Disciplina (validação no domínio/back-end; reforço opcional no banco).
- Exclusão lógica: ativo = false. Desativar um Material usado pelo Monitor Virtual ou pela prática remove seus chunks do índice vetorial.
- Prática (proposta): só em Disciplina com possuiProva = true.
- disponivelParaPratica = true exige tipo LISTA_EXERCICIOS ou PROVA_ANTIGA e autorizacaoProfessor = true (CHECK no banco) e Disciplina com prova (back-end).
- Sem histórico de chat (thread_id) nem de prática (sessaoId) no banco relacional no MVP.
- Monitor humano (candidatura, agenda, mensagens, avaliações) fora do escopo.

NEURON — Diagrama de Classes: Backend Spring Boot (camadas)

Controller -> Service -> Repository por domínio, integração com o serviço de IA via AiClient. API pública em /api/v1.

classe nova (proposta) item laranja* = proposta a validar

Controller

«RestController» /api/v1/auth
AuthController
+login(LoginRequest): TokenResponse
+logout(): void
+me(): UsuarioResponse

«RestController» /api/v1/feedback
FeedbackController
+enviar(FeedbackRequest): 201
+listarAdmin(): List<FeedbackResponse>
GET /admin/feedback (ADMINISTRADOR)

«RestController» /api/v1
DisciplinaController
+listarPorPeriodo(periodoId)
+detalhar(id): DisciplinaResponse
+listarAssuntos(disciplinaId)
+criar/atualizar/desativar «ADMIN»

«RestController» /api/v1/pesquisa
PesquisaController
+pesquisar(q): ResultadoPesquisa

«RestController» /api/v1
MaterialController
+listar(disciplinaId, assuntoId?)
+criar(MaterialRequest) «ADMIN»
+atualizar(id, MaterialRequest)
+desativar(id) (exclusão lógica)

«RestController» /api/v1/pratica
PraticaController
+iniciarSessao(PraticaRequest)
+responder(sessaoId, RespostaRequest)

«RestController» /api/v1/monitor
MonitorVirtualController
+chat(ChatRequest): text/event-stream
streaming SSE (POST)

Service

«Service»
AuthService
+autenticar(email, senha): Token
+encerrarSessao(): void
+usuarioAutenticado(): Usuario

«Service»
FeedbackService
+registrar(usuarioId, FeedbackRequest)
+listarTodos(): List<FeedbackPlataforma>
-validarNota(nota): void (1..5)

«Service»
DisciplinaService
+listarPorPeriodo(periodoId)
+detalhar(id): Disciplina
+criar(DisciplinaRequest) (possuiProva*)
+desativar(id): void

«Service»
PesquisaService
+pesquisar(termo): ResultadoPesquisa
busca comum, separada do RAG

«Service»
MaterialService
+criar(MaterialRequest)
+atualizar(id, MaterialRequest)
+desativar(id): void
-validarAssuntoDaDisciplina()
-validarRegrasDePratica()

«Service»
PraticaService
+iniciarSessao(PraticaRequest)
+corrigir(sessaoId, RespostaRequest)
-exigirDisciplinaComProva() 409
DISCIPLINA_SEM_PROVA (sessaoId efêmero)

«Service»
MonitorVirtualService
+conversar(ChatRequest, usuario)
-validarDisciplina(disciplinaId)
-aplicarRateLimit(usuario)
gera/propaga thread_id

Repository (Spring Data JPA) -> PostgreSQL

«Repository» JpaRepository
UsuarioRepository
+findByEmail(String): Optional<Usuario>

«Repository» JpaRepository
FeedbackRepository
+findAllByOrderByCriadoEmDesc()

«Repository» JpaRepository
DisciplinaRepository
+findByPeriodoIdAndAtivoTrue(id)
+findByIdAndAtivoTrue(id)

«Repository» JpaRepository
PesquisaRepository
+buscarDisciplinas(q)
+buscarAssuntos(q)
+buscarMateriais(q)

«Repository» JpaRepository
MaterialRepository
+findByDisciplinaIdAndAtivoTrue(id)
+findByDisciplinaIdAndAssuntoId...()

Transversal / Integração

«Service» pacote security
TokenService
+gerarToken(Usuario): String
+validarToken(String): boolean
+extrairUsuario(String): Usuario
Estratégia final de autenticação (JWT) ainda a aprovar.

«DTO» isolam o contrato HTTP das entidades
DTOs (records)
LoginRequest, TokenResponse, UsuarioResponse
DisciplinaResponse (possuiProva*)
MaterialRequest, MaterialResponse,
MaterialResumoResponse (+campos de prática*)
ChatRequest, ResultadoPesquisa
PraticaRequest, RespostaRequest, QuestaoResponse
FeedbackRequest, FeedbackResponse

«ControllerAdvice»
GlobalExceptionHandler
+tratar(Exception): ErroResponse
ErroResponse {status, codigo, mensagem, timestamp}. 400, 401, 403, 404, 409, 429, 503. DisciplinaSemProvaException -> 409.

«Component» integration.ai (único ponto de contato com o FastAPI)
AiClient
-baseUrl: String = ${AI_SERVICE_URL}
-http: WebClient
+chat(AiChatRequest): Flux<AiEvento>
+indexar(AiIndexRequest): void
+remover(materialId): void
+reindexar(materialId): void
+montarQuestoes(AiPracticeRequest): List<AiQuestao>
+corrigir(AiRespostaRequest): AiCorrecao
Services do domínio não conhecem URL interna nem formato do framework Python. Indisponível -> 503. Endpoints /internal/v1 não são expostos ao navegador. DTOs: AiChatRequest, AiIndexRequest, AiPracticeRequest, AiSourceResponse.*

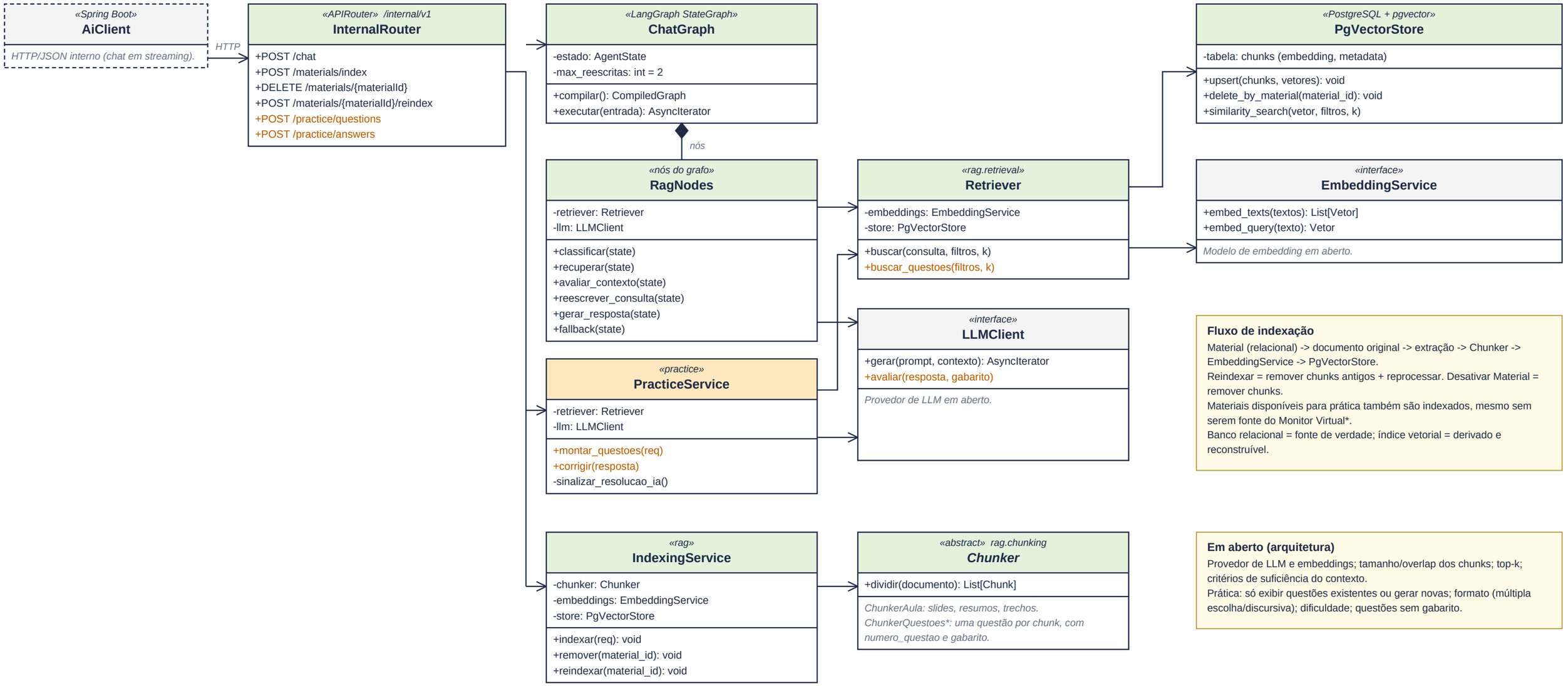
indexar / remover / reindexar
*questões e correção**
chat (SSE)

Periodo e Assunto seguem o mesmo padrão de Disciplina (PeriodoController/Service/Repository e AssuntoController/Service/Repository). Endpoints /admin/ exigem perfil ADMINISTRADOR.*
PraticaService valida disciplina.possuiProva antes de chamar a IA. Migrations versionadas com Flyway (db/migration). Feedback não passa pelo serviço de IA.*

NEURON — Diagrama de Classes: Serviço de IA (FastAPI + LangChain/LangGraph + pgvector)

API interna /internal/v1 (chamada apenas pelo Spring Boot): Monitor Virtual (RAG), prática com questões* e indexação de materiais.

classe nova (proposta) item laranja* = proposta a validar

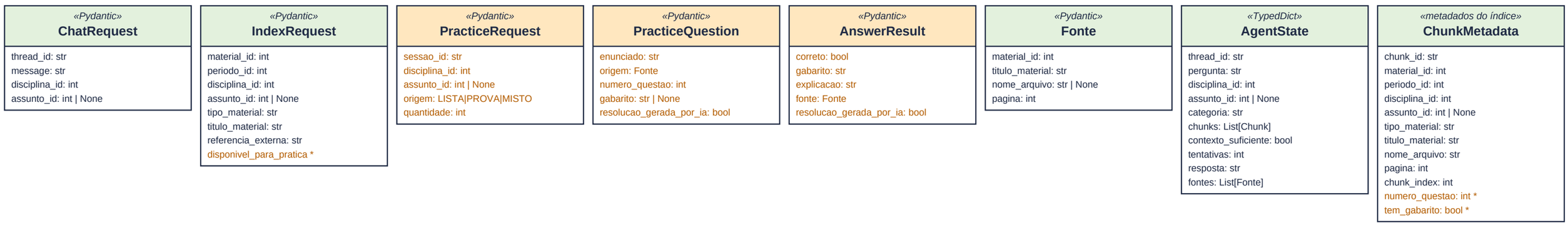


Fluxo de indexação

Material (relacional) -> documento original -> extração -> Chunker -> EmbeddingService -> PgVectorStore.
Reindexar = remover chunks antigos + reprocessar. Desativar Material = remover chunks.
Materiais disponíveis para prática também são indexados, mesmo sem serem fonte do Monitor Virtual*.
Banco relacional = fonte de verdade; índice vetorial = derivado e reconstruível.

Em aberto (arquitetura)

Provedor de LLM e embeddings; tamanho/overlap dos chunks; top-k; critérios de suficiência do contexto.
Prática: só exibir questões existentes ou gerar novas; formato (múltipla escolha/discursiva); dificuldade; questões sem gabarito.

Fluxo do chat: classificar -> recuperar (top-k filtrado) -> contexto suficiente? -> gerar com fontes | reescrever (máx. 2x) -> fallback sem inventar.

Schemas (Pydantic) e estruturas auxiliares